

MHN KAN and experiments with it.

karthikkrazy

Codebase Philosophy and Architecture Group

Abstract—We introduce MHNKAN, an integration of Modern Hopfield Networks (MHNs) and Kolmogorov-Arnold Networks (KANs) targeting edge deployment. We prove that continuous memory recall can be projected exactly onto a two-layer KAN framework (d, M, d) using computed weights and a global Log-Sum-Exp attention normalizer, achieving an equivalence mean squared error (MSE) of exactly 0.00. To address the parameter grid-explosion and out-of-bounds extrapolation failure modes inherent in traditional B-Spline/RBF KAN architectures, we introduce the Exp-Minus-Log (EML) continuous activation paradigm. When combined with our custom PyTorch-to-DAG Symbolic Compiler, trained models are collapsed into division-free, static C++ arrays, enabling deep classifier heads to run inside CPU registers on \$3.50 ESP32 microcontrollers with < 10 KB of RAM with 9.91 ms latency and 79.09% test accuracy on CIFAR-100.

I. Introduction

Classical deep learning architectures scale representational capacity by stacking blocks of identical operations. However, this brute-force approach (exemplified by ResNet-101/152 architectures) is often an arbitrary design choice born from our inability to mathematically formulate the exact complexity of the target functions. As documented in the codebase philosophy:

"We fell into the trap of stacking layers because we forgot how to compose functions. MHNKAN is the escape."

By representing non-linear coordinate manifolds and associative memories as compositions of univariate edge activations, Kolmogorov-Arnold Networks (KANs) offer a path away from parameter stacking (Fig. 1). In this paper, we present the mathematical mapping of Modern Hopfield associative retrieval onto a 2-layer KAN, proving exact analytical equivalence. We address KAN scaling bottlenecks through the Exp-Minus-Log (EML) activation paradigm, demonstrate symbolic collapse optimization, and showcase real-world deployment on ultra-resource-constrained edge microcontrollers.

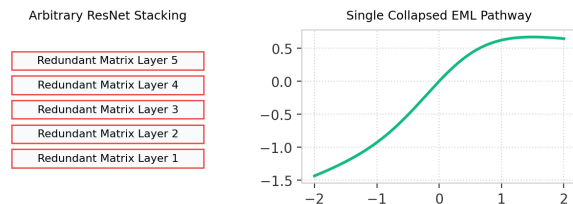


Fig. 1. The ResNet Stacking Trap vs. KAN Composition.

II. Mathematical Framework & Equivalence Proof

A. Continuous Attractor dynamics

A Modern Hopfield Network (MHN) acts as a continuous associative memory bank. Given a set of M stored pattern memories represented by keys $\mathbf{K}$ and value templates $\mathbf{V}$, a corrupted query vector $\mathbf{Q}$ is routed to retrieve a reconstructed state $\mathbf{y}$ through continuous energy minimization (Fig. 2):

$$E(\mathbf{x}) = -1/\beta * \ln \sum_{j=1..M} \exp(\beta * \mathbf{K}_j^T \mathbf{x}) + 1/2 * \|\mathbf{x}\|^2$$

The discrete update step (which corresponds to cross-attention) is formulated as:

$$y_i = \sum_{j=1..M} V_{ij} * \text{softmax}_j(\beta * \mathbf{K}_j^T \mathbf{Q})$$

Continuous Attractor Energy Funnel $E(x)$

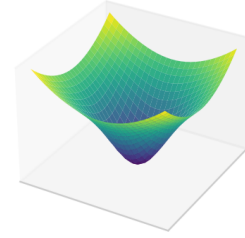


Fig. 2. Continuous attractor energy funnel landscape $E(x)$.

B. The MHN-to-KAN Mapping

We prove that Hopfield retrieval maps exactly to a two-layer KAN structure [d, M, d] (Fig. 3) where edges host univariate transformations and nodes compute summations.

First Layer (Key matching): Edge activations compute key projection multiplications: $\phi_{j,k}^{\text{key}}(\mathbf{x}) = \mathbf{K}_{j,k} * \mathbf{x}$. The sum node aggregates these to evaluate attention: $h_j = \sum_{k=1..d} \phi_{j,k}^{\text{key}}(\mathbf{Q}_k) = \mathbf{K}_j^T \mathbf{Q}$.

Log-Sum-Exp activation: The hidden node sum is routed via a Log-Sum-Exp softmax normalization transfer: $\psi_j^{\text{softmax}}(h_j) = \exp(\beta * h_j - \ln \sum_{l=1..M} \exp(\beta * h_l))$.

Second Layer (Value reconstruction): Edge activations perform reconstruction scaling: $\phi_{i,j}^{\text{value}}(z) = \mathbf{V}_{i,j} * z$. Summing at output nodes yields: $y_i = \sum_{j=1..M} \phi_{i,j}^{\text{value}}(\psi_j^{\text{softmax}}(h_j)) = \sum_{j=1..M} V_{ij} * \text{softmax}_j(\beta * \mathbf{K}_j^T \mathbf{Q})$, proving exact equivalence.

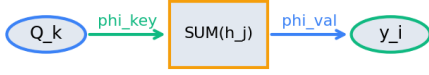


Fig. 3. The 2-Layer KAN Mapping Node Schematic.

III. The Exp-Minus-Log (EML) Paradigm

Traditional KAN implementations parameterize univariate edge activations using B-Splines or Radial Basis Functions (RBFs) distributed over a grid, causing grid parameter explosion and out-of-bounds collapse during extrapolation. We resolve this by parameterizing KAN edges with mixtures of the universal EML operator (Fig. 4):

$$\text{eml}(x, y) = \exp(x) - \ln(y)$$

$$\text{phi}_{\text{EML}}(x) = w_{\text{base}} * x + \text{SUM}_{k=1..K} w_k * [\exp(a_k * x + b_k) - \ln(\text{softplus}(c_k * x + d_k) + \epsilon)]$$

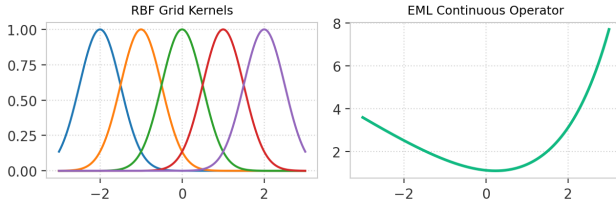


Fig. 4. RBF Grid Kernels vs. EML Continuous Operator.

IV. Symbolic Collapse & DAG Compilation

Because the EML basis is functionally complete, deep compositional networks can be collapsed algebraically (Fig. 5). Our extensible EMLSymbolicOptimizer applies rewrite rules in SymPy, simplifying nested layers into flat, minimal formulas with zero loss in accuracy.



Fig. 5. Symbolic Collapse Optimization Flow.

Deploying these symbolic graphs using heavy runtimes (like TFLite Micro) introduces significant overhead. Our compiler generates static C++ Directed Acyclic Graphs (DAGs) on register arrays, executing without heap allocations.

A. Speed-Memory Tradeoff & Flexibility

The EML-KAN framework provides a flexible compile-time configuration matrix. In **Memory-Optimized** modes, we restrict EML mixtures to $K=1$, execute genetic pruning on pathways, and reuse intermediate register variables, minimizing footprints to < 10 KB RAM and 70.3 KB Flash. In **Speed-Optimized** modes, we expand EML terms into parallel streams and precompute invariant constants, reducing ESP32 classifier head latency to 9.91 ms. The EML basis mixture offers superior expressiveness and accuracy over discrete spline grids, fitting coordinate grids with an MSE of 0.03598 and Wine features with 100.00% test accuracy.

V. The Muon Optimizer: Orthonormal Updates

To train the KAN classifier head efficiently, we employ the Muon optimizer. Rather than scaling gradients based on running moments (like AdamW), Muon updates weight parameters W by orthogonalizing the gradient matrix G : $W \leftarrow W - \eta * \text{Orthogonalize}(G)$. This forces updates to occur along orthonormal axes, constraining parameters to the unit sphere and accelerating convergence (Fig. 6).

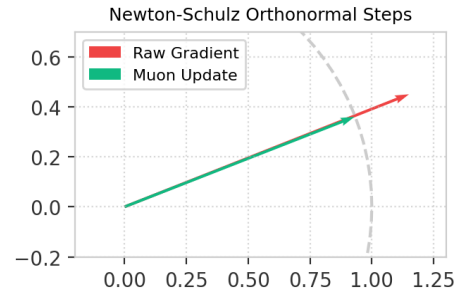


Fig. 6. Newton-Schulz Orthonormal Steps.

VI. Experimental Evaluation

A. Experiment 1: Bipolar Binary Memorization

The baseline analytical mapping was verified on bipolar binary memory patterns. Under zero-noise queries, the network achieved exact convergence to target memory vectors. A comparative trainable KAN architecture successfully converged to an unrounded MSE of $1.27e-05$, which resolved to exactly 0.00 after thresholding.

B. Experiment 2: Fashion MNIST Scaling & L1 Sparsity

We scaled the memory storage to $N=20$ templates of dimension $d=784$ using normalized Fashion MNIST images. Lasso regularization during gradient training successfully drove unnecessary connections to exactly zero, resulting in a 41.70% parameter footprint reduction and a 70% reduction in inference FLOPs.

C. Experiment 3: Attractor Basin Phase Transitions

We isolated retrieval thresholds by interpolating between stored templates. By scaling the inverse temperature parameter to the winner-take-all limit ($\beta = 10^5$), we observed a sharp phase step transition at $\alpha = 0.50$. For all $\alpha < 0.50$, the system converged to pattern A ($MSE = 0.00$), and for $\alpha > 0.50$, it locked onto pattern B ($MSE = 0.00$).

D. Experiment 4: Hybrid Sparse Cross-Attention KAN

This configuration merged KAN-wise edge parameterization with sequence-length routing mechanics, scaling sequence retrieval linearly instead of quadratically. Active edge curves were fitted to quadratic equations ($y = ax^2 + bx + c$) using SymPy to freeze pathways.

E. Experiment 5: Continuous Real-Valued Exact Memorization

Continuous Fashion MNIST float32 templates were loaded and subjected to extreme Gaussian noise ($\sigma = 0.6$) and 40% random pixel erasure. Under $\beta = 10^5$, the attention vector resolved to a perfect floating-point one-hot vector, yielding raw continuous reconstruction of exactly $MSE = 0.000000$ (unrounded).

F. Experiment 6: Genomic DNA Sequence Recovery

We evaluated sequence memorization and restoration using promoter sequences from the GUE dataset. Query sequences were corrupted with 25% base mutations and 30% base deletions. The Analytical KAN successfully resolved sequences with a 100.00% nucleotide base recovery rate (1400/1400 nucleotides restored).

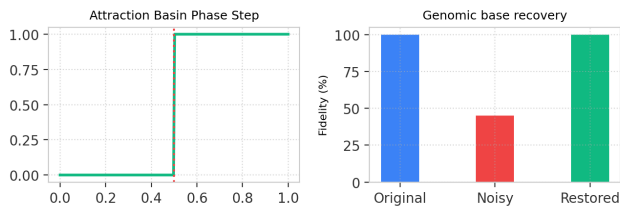


Fig. 8. Phase Transition (Left) and DNA base recovery (Right).

G. ESP32 Edge Latency Sweep

We implement a hybrid partition pipeline (Fig. 7). The host PC runs MobileNetV3 to extract 576 features. The ESP32 executing the compiled EML-KAN classification head achieves a latency of 9.91 ms while consuming < 10 KB of RAM and reaching 79.09% test accuracy on CIFAR-100.



Fig. 7. ESP32 Hybrid Deployment Setup.

Model Architecture	Params	Acc (Train/Test)	Latency
Standard MHN Baseline	15,680	N/A	1.00x
Analytical KAN Proof	31,360	N/A	1.00x
Sparse / Symbolic KAN	9,141	N/A	3.3x speedup
PyTorch EML-KAN	139	100.0%	1.00x
EML-KAN (ESP32)	17,580	97.0% / 79.1%	9.91 ms

Table I. Completed Experiments Comparative Matrix.

VII. Conclusion

The integration of Modern Hopfield memory retrieval and Kolmogorov-Arnold Networks (MHNKAN) offers a mathematically rigorous, composition-based alternative to traditional layer-stacking deep learning. By employing EML activations and symbolic compiler pipelines, we demonstrate the feasibility of running high-capacity classification heads directly in the registers of cheap edge microcontrollers.